

Design and Analysis of Algorithms: CI43

Unit-II

Graphs & Divide and Conquer

1. Graph Connectivity

- Graph connectivity refers to whether all vertices (nodes) in a graph are connected in some way. A graph can be categorized into:
- **Connected Graph:** A graph is connected if there is a path between any two vertices.
- **Disconnected Graph:** A graph is disconnected if at least one pair of vertices has no path between them.
- **Strongly Connected (for Directed Graphs):** A directed graph is strongly connected if there is a directed path from any vertex to every other vertex.
- **Weakly Connected (for Directed Graphs):** A directed graph is weakly connected if replacing all directed edges with undirected edges results in a connected graph.

Graph Traversal

Graph traversal algorithms explore the vertices and edges of a graph. The two main types are:

- **Breadth-First Search (BFS)**
- **Depth-First Search (DFS)**

2.1 Breadth-First Search (BFS): Exploring a Connected Component

- BFS explores all nodes at the present depth before moving deeper. It uses a **queue (FIFO)** structure.

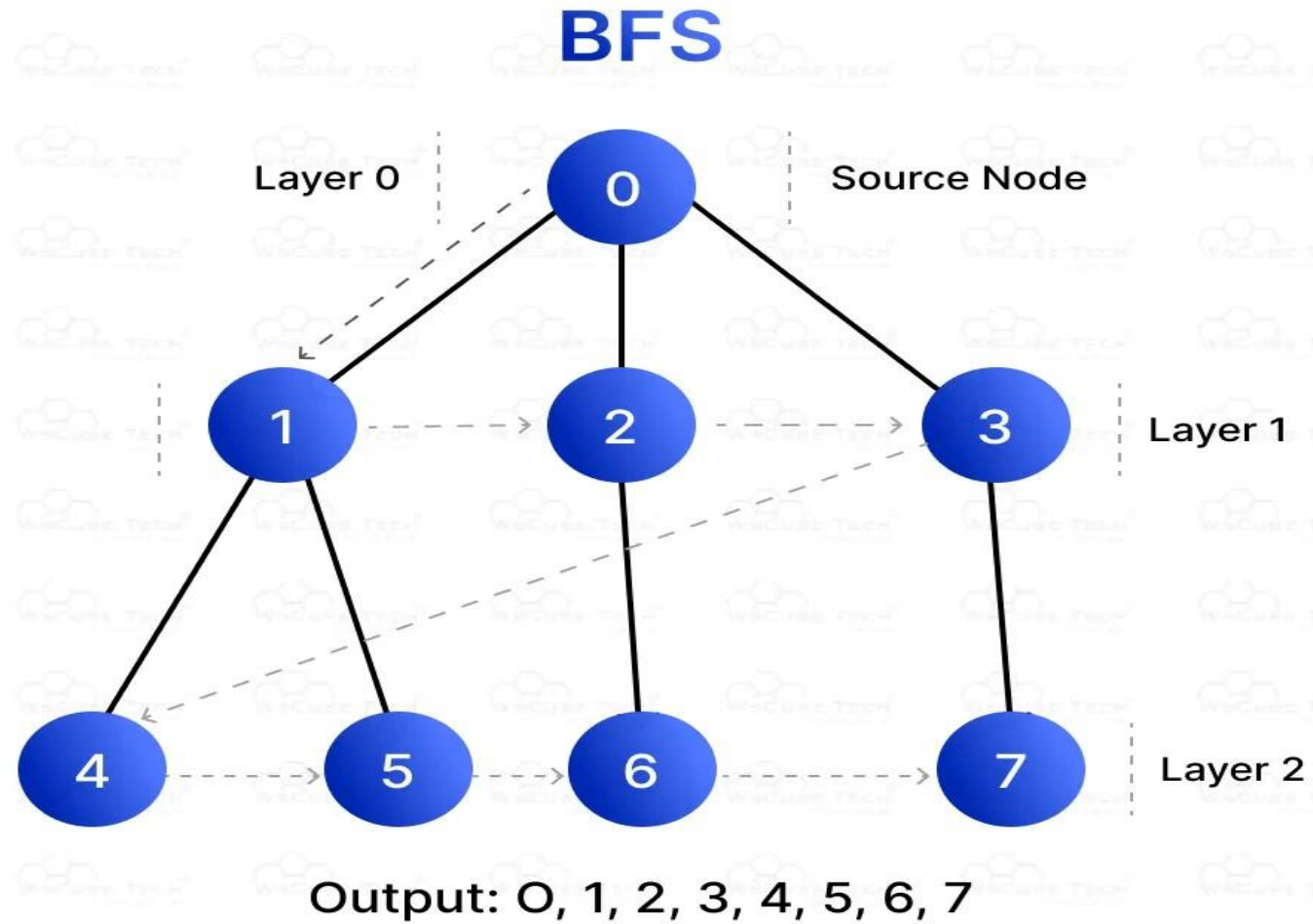
BFS Algorithm:

- Start from a source node.
- Visit all its neighbors before moving to their neighbors.
- Use a queue to keep track of visited nodes.

BFS (Breadth-First Search) is a graph traversal algorithm that explores nodes level by level, starting from a given source node. It visits all neighboring nodes at the current depth (or level) before moving on to nodes at the next depth level. BFS is commonly used for traversing or searching tree.

- **Approach:** BFS explores the graph level by level, starting from the source node. It visits all the nodes at one level before moving to the next. BFS uses a queue to keep track of nodes that need to be explored.
- **How it works:** From the source node, BFS moves to all its direct neighbors (Layer 1), then moves to the next layer (Layer 2), continuing until all nodes are visited.

Breadth-First Search (BFS) Implementation *DEVOTION TO ENLIGHTENMENT*



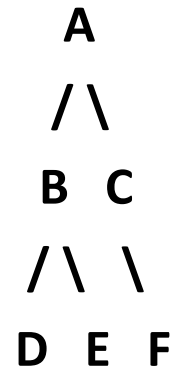
Traversal Order:

- Starting from node 0:
- Layer 0: Visit node 0 (source node).
- Layer 1: Visit nodes 1, 2, 3 (neighbors of node 0).
- Layer 2: Visit nodes 4, 5, 6, 7 (neighbors of nodes 1, 2, 3).
- Output: 0, 1, 2, 3, 4, 5, 6, 7.

Applications of BFS:

- Finding the shortest path in an unweighted graph.
- Checking graph connectivity.
- Solving mazes and puzzles.

BFS is a level-order traversal where we visit all **neighboring** nodes before going deeper. It uses a **queue (FIFO: First In, First Out)**.



- **BFS Traversal (Starting from A)**
- Start at **A** → Visit **A**, add its neighbors (**B, C**) to the queue.
- Dequeue **B** → Visit **B**, add its neighbors (**D, E**) to the queue.
- Dequeue **C** → Visit **C**, add its neighbor (**F**) to the queue.
- Dequeue **D** → Visit **D** (no more neighbors).
- Dequeue **E** → Visit **E** (no more neighbors).
- Dequeue **F** → Visit **F** (no more neighbors).

- **Final BFS Order: A → B → C → D → E → F**

Queue Operations

Step	Queue (FIFO)	Processing Node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F]	C
4	[E, F]	D
5	[F]	E
6	[]	F

2.2 Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It uses a **stack (LIFO)** or recursion.

DFS Algorithm:

Start from a source node.

Recursively explore each unvisited neighbor.

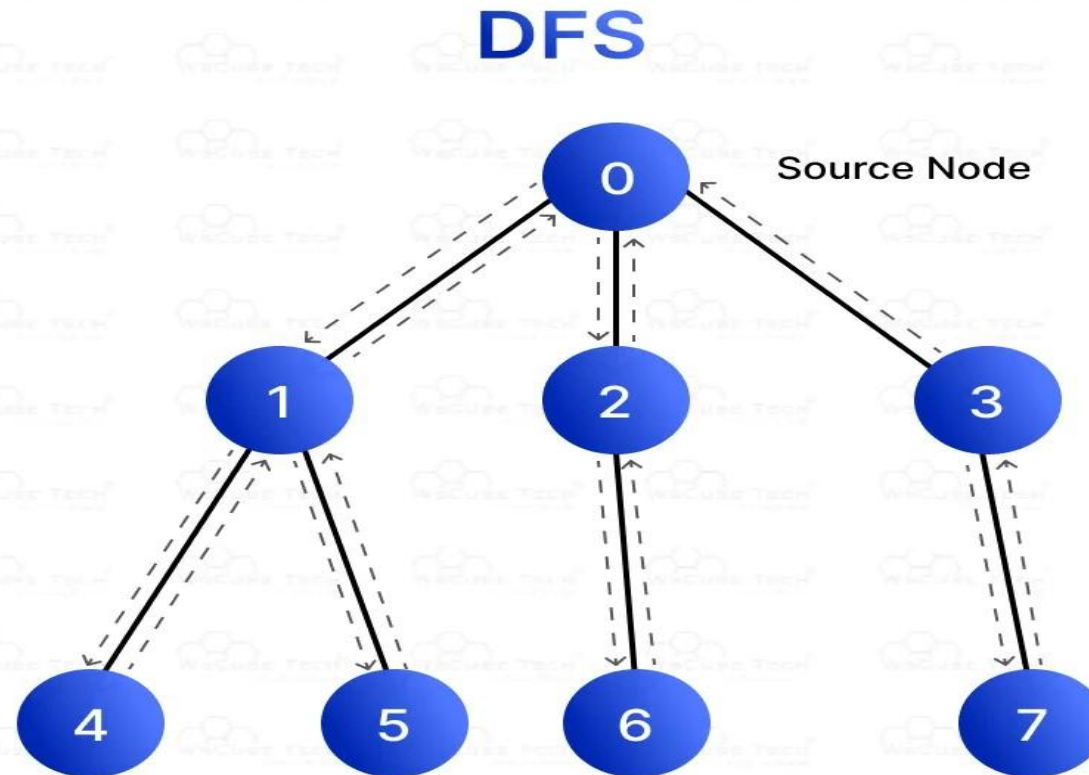
Backtrack when no unvisited neighbors are left.

- **Approach:** [DFS](#) explores as deep as possible along a branch before backtracking. It uses a [stack](#) (or recursion) to keep track of nodes and backtracks when it reaches a dead end.
- **How it works:** Starting from the source node, DFS goes deeper into the graph, visiting one branch of the graph first before [backtracking](#) to explore others.

Applications of DFS:

- 1. Topological sorting.**
- 2. Detecting cycles in graphs.**
- 3. Finding strongly connected components**

Depth-First Search (DFS)

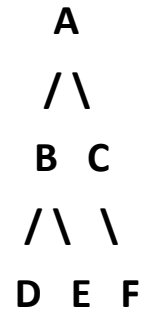


Traversal Order:

- Starting from node 0:
- Visit node 0 → Visit node 1 → Visit node 4 (deepest) → Backtrack to node 1 → Visit node 5 → Backtrack to node 0 → Visit node 2 → Visit node 6 → Backtrack to node 2 → Visit node 3 → Visit node 7.
- **Output:** 0, 1, 4, 5, 2, 6, 3, 7

- **Definition:**

DFS explores as **deep as possible** along a branch before backtracking. It uses a **stack (LIFO: Last In, First Out)**.



- **DFS Traversal (Starting from A)**

- Start at **A** → Visit **A**, push neighbors (**B**, **C**) to the stack.
- Pop **C** → Visit **C**, push neighbor (**F**).
- Pop **F** → Visit **F** (no more neighbors).
- Pop **B** → Visit **B**, push neighbors (**D**, **E**).

- Pop **E** → Visit **E** (no more neighbors).

Step-by-Step DFS Traversal (Using Stack)

- Start at A
- Visit A
- Push its neighbors (B, C) onto the stack.
- Stack: [B, C]
- Pop C from stack (LIFO order)
- Visit C
- Push its neighbor F onto the stack.

- Stack: [B, F]

- F has no children, so nothing is pushed.

- Stack: [B]

- Pop B from stack

- Visit B

- Push its neighbors (D, E) onto the stack.

- Stack: [D, E]

- Pop E from stack

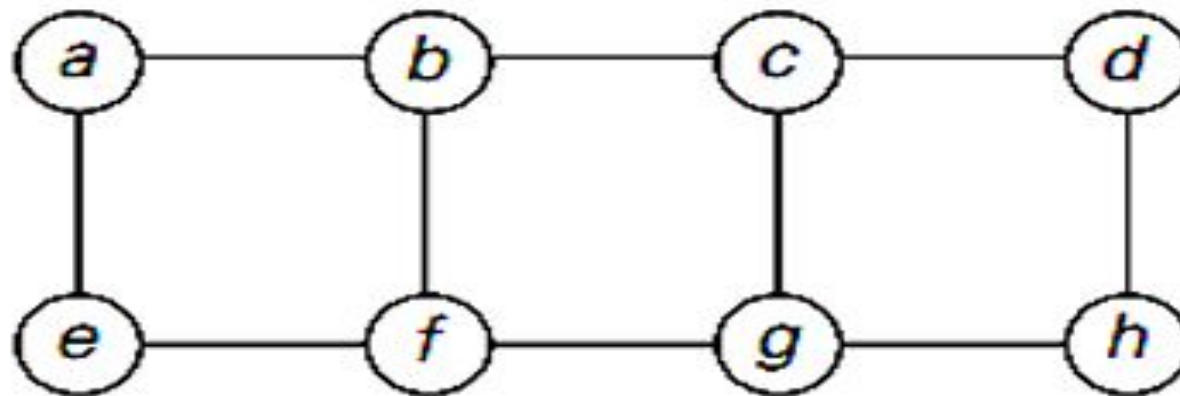
- Visit E

- E has no children, so nothing is pushed.

Problems on BFS And Solution

Consider the graph given in Fig

1. Starting at vertex 'b' and resolving ties by the vertex alphabetical order, traverse the graph by Breadth First Search (BFS) and construct the corresponding Breadth First Search Spanning Tree.
2. Illustrate the Queue operations for performing BFS Traversal to obtain BFS Spanning Tree.
3. Specify all possible BFS Traversal sequences.



- **BFS Traversal (Starting from 'b')**
- **Start at 'b', enqueue → Queue: [b]**
- **Visit neighbors: a, c, f (in alphabetical order) → Queue: [a, c, f]**
- **Dequeue 'a', visit neighbors: e (already visited b) → Queue: [c, f, e]**
- **Dequeue 'c', visit neighbors: d, g → Queue: [f, e, d, g]**
- **Dequeue 'f', no new neighbors → Queue: [e, d, g]**
- **Dequeue 'e', no new neighbors → Queue: [d, g]**
- **Dequeue 'd', visit neighbor: h → Queue: [g, h]**
- **Dequeue 'g', no new neighbors → Queue: [h]**
- **Dequeue 'h', no new neighbors → Queue: [] (empty)**

- **Final BFS Traversal Order:**

- **Step 2: BFS Spanning Tree**

- The BFS Spanning Tree includes edges that represent the first discovery of each node.

- **Root: b**

Tree edges:

$b \rightarrow a$

$b \rightarrow c$

$b \rightarrow e$

$b \rightarrow f$

$c \rightarrow d$

$e \rightarrow h$

$c \rightarrow g$

The spanning tree structure:

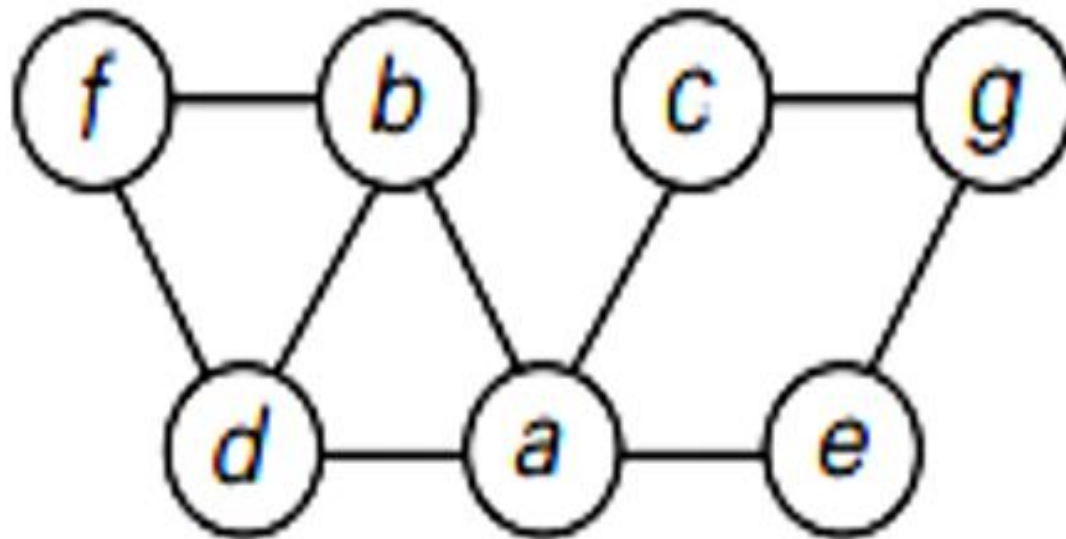


- **Step 4: Possible BFS Traversal Sequences**
- Since BFS depends on tie-breaking when selecting neighbors, other BFS sequences can occur depending on how ties are resolved. The given order follows **alphabetical order**, but alternative tie-breaking might yield sequences such as:
 - **b, a, c, e, f, g, d, h**
 - **b, a, c, f, e, d, g, h**
 - **b, c, a, e, f, d, g, h**
- Each sequence depends on which neighbor is processed first when multiple choices are available.

Problems on DFS and Solution

Consider the graph given in Fig.

1. Starting at vertex 'a' and resolving ties by the vertex alphabetical order, traverse the graph by Depth First Search and construct the corresponding Depth First Search Spanning Tree.
2. Show the traces of the traversal stack for DFS and specify the order in which vertices were pushed onto and popped off the stack.
3. Identify back edges, if any. "All DFS forests will have same number of tree edges and back edges", Is this statement true? Justify the answer



Adjacency List Representation

- From the image, we extract the edges:

a - b, a - c, a - d, a - e

b - f, b - d

c - g

So the adjacency list is:

$a \rightarrow \{b, c, d, e\}$

$b \rightarrow \{a, d, f\}$

$c \rightarrow \{a, g\}$

$d \rightarrow \{a, b\}$

$e \rightarrow \{a\}$

$f \rightarrow \{b\}$

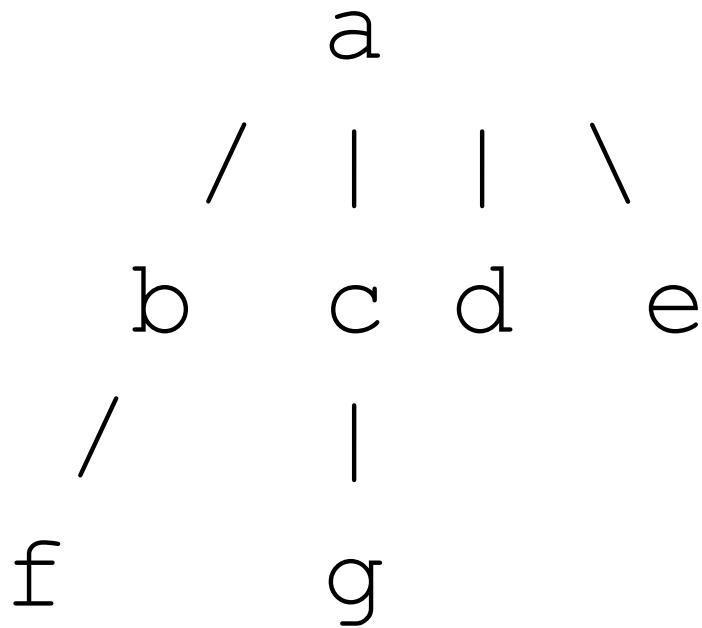
$g \rightarrow \{c\}$

Contd..

1. Start at a.
2. Visit b (first available neighbor in alphabetical order).
3. From b, visit f (only neighbor).
4. Backtrack to b, no more unvisited neighbors.
5. Backtrack to a.
6. Visit c (next available neighbor).
7. From c, visit g (only neighbor).
8. Backtrack to c, no more unvisited neighbors.
9. Backtrack to a.
10. Visit d (next available neighbor).
11. Backtrack to a, no more unvisited neighbors.
12. Visit e (next available neighbor).
13. Backtrack to a, DFS complete.

DFS Spanning Tree:

..



Tree Edges: These are the edges that form the DFS tree structure:
(a, b), (b, f), (a, c), (c, g), (a, d), (a, e)

DFS Stack Tracing

Step	Action	Stack	Notes
1	Push a	[a]	Start DFS
2	Push b	[a, b]	Visit 'b'
3	Push f	[a, b, f]	Visit 'f'
4	Pop f	[a, b]	No more neighbors
5	Pop b	[a]	No more neighbors
6	Push c	[a, c]	Visit 'c'
7	Push g	[a, c, g]	Visit 'g'
8	Pop g	[a, c]	No more neighbors
9	Pop c	[a]	No more neighbors
10	Push d	[a, d]	Visit 'd'
11	Pop d	[a]	No more neighbors
12	Push e	[a, e]	Visit 'e'
13	Pop e	[a]	No more neighbors
14	Pop a	[]	DFS Complete

Contd..

Step 4: Identify Back Edges

Back edges occur when a node links back to an **already visited ancestor** (except the direct parent).

No back edges exist in this DFS traversal.

The given graph is a tree (acyclic), so there are no cycles that would introduce back edges.

Are all DFS forests the same?

The statement "All DFS forests will have the same number of tree edges and back edges" is true because:

The number of tree edges (edges used in the DFS spanning tree) is always $n-1$ in a connected graph.

Back edges depend on cycles and remain consistent in all DFS forests.

Thus, while traversal order may change due to different starting points or tie-breaking, the total count of tree edges and back edges remains unchanged.

Criteria	BFS (Breadth-First Search)	DFS (Depth-First Search)	T
Traversal Method	Explores all neighbors at the current level before moving to the next level.	Explores as deep as possible along one branch before backtracking.	
Data Structure Used	Queue (FIFO)	Stack (LIFO) or recursion stack.	
Time Complexity	$O(V + E)$ (V = vertices, E = edges)	$O(V + E)$ (V = vertices, E = edges)	
Space Complexity	$O(V)$ (due to queue storing nodes at the current level)	$O(V)$ (due to recursion or explicit stack)	
Best for	Finding the shortest path in an unweighted graph.	Exploring all possible paths or solving puzzles like mazes.	
Use Case	- Shortest path in unweighted graphs (like social networks, web crawlers). - Level-order traversal of trees.	- Solving maze or puzzle problems. - Detecting cycles in graphs, topological sorting.	
Performance on Deep Graphs	May require more memory for wide/deep graphs, as the queue holds many nodes.	Efficient for deep graphs, as it traverses a single path at a time.	
Performance on Wide Graphs	Efficient for shallow/wide graphs, as it processes level by level.	May require more memory for wide graphs, as the recursion stack can grow large.	
Shortest Path	Guaranteed to find the shortest path in an unweighted graph.	Not guaranteed to find the shortest path, but explores all paths.	
Pathfinding in Weighted Graphs	Not suitable for weighted graphs.	Not suitable for weighted graphs (Dijkstra's or Bellman-Ford should be used).	
Cycle Detection	Can detect cycles, but less commonly used for it.	Useful for detecting cycles in directed or undirected graphs.	
Recursive/Iterative	Iterative (uses a queue)	Can be recursive (via recursion stack) or iterative (using an explicit stack).	
Graph Representation	Can be used with both adjacency lists and matrices.	Can be used with both adjacency lists and matrices.	
Tree Traversal	Used for level-order traversal in trees.	Used for pre-order, in-order, and post-order traversal in trees.	
Backtracking	No backtracking.	Uses backtracking to explore new branches.	
Algorithm Type	Greedy approach (explores level by level).	Backtracking approach (explores depth first).	

- **DFS vs BFS: Which is Faster?**
- Neither DFS nor BFS is inherently faster than the other; both have the same time complexity of $O(V + E)$, where V is the number of vertices and E is the number of edges.
- However, BFS is faster for finding the shortest path in unweighted graphs, while DFS can be faster when searching deeper paths without concern for the shortest path. The speed depends on the specific problem and the structure of the graph.

- Application of DFS and BFS :

DFS Applications	BFS Applications
Cycle detection in graphs	Finding the shortest path in unweighted graphs
Finding connected components in a graph	Level-order traversal in trees
Solving puzzles like Sudoku, N-Queens (backtracking)	Web crawling (breadth-first)
Topological sorting in Directed Acyclic Graphs (DAGs)	Social network analysis (finding the shortest path between users)
Finding strongly connected components (Tarjan's/Kosaraju's algorithm)	Finding nodes at a specific level in a graph or tree
Maze solving (exploring all paths)	Minimum moves in a game or puzzle (e.g., 8-puzzle)
Web crawling (depth-first)	Friend suggestion algorithms in social networks
Tree traversal (pre-order, in-order, post-order)	Finding the shortest path in a maze
Pathfinding in games (exploring all paths)	Level-by-level exploration in AI game design

3. Implementing Graph Traversal Using Queues and Stacks

3. Implementing Graph Traversal Using Queues and Stacks

- **BFS uses a queue (FIFO)** to explore nodes level by level.
- **DFS uses a stack (LIFO) or recursion** to explore as far as possible before backtracking
- **BFS vs DFS Comparison:**

Feature	BFS (Queue)	DFS (Stack/Recursion)
Data Structure	Queue (FIFO)	Stack (LIFO)
Traversal Order	Level-wise	Depth-wise
Uses	Shortest path, network broadcasting	Cycle detection, backtracking
Time Complexity	$O(V + E)$	$O(V + E)$

4. Directed Acyclic Graphs (DAG) and Topological Sorting

A Directed Acyclic Graph (DAG) is a directed graph with no cycles.

Topological Sorting

- A linear ordering of vertices such that for every directed edge ($u \rightarrow v$), u appears before v .
- Used in task scheduling, dependency resolution, and course prerequisites.

Topological Sorting Algorithm (Using DFS):

```
def topological_sort(graph, node, visited, stack):  
    visited.add(node)  
    for neighbor in graph[node]:  
        if neighbor not in visited:  
            topological_sort(graph, neighbor, visited, stack)  
    stack.append(node)
```

Applications of Topological Sorting:

- Task scheduling (e.g., job scheduling in an operating system)
- Course prerequisite management
- Dependency resolution (e.g., package installation order)

5.Merge Sort Algorithm (Divide and Conquer)

Merge Sort Algorithm (Divide and Conquer):

Merge Sort is a **divide and conquer** algorithm that:

- **Divides** the array into halves.
- **Recursively sorts** both halves.
- **Merges** the sorted halves.

5.Merge Sort Algorithm (Divide and Conquer)

• Merge Sort **Algorithm Steps**

- **Divide:** Split the array into two halves.
- **Conquer:** Recursively sort both halves.
- **Merge:** Combine the sorted halves back into a single sorted array.

5.Merge Sort Algorithm (Divide and Conquer)

MergeSort(A, left, right)

- 1. If $\text{left} < \text{right}$:**
- 2. Find the middle index: $\text{mid} = (\text{left} + \text{right}) / 2$**
- 3. Recursively call MergeSort on first half: MergeSort(A, left, mid)**
- 4. Recursively call MergeSort on second half: MergeSort(A, mid + 1, right)**
- 5. Merge the two sorted halves: Merge(A, left, mid, right)**

5.Merge Sort Algorithm (Divide and Conquer)

Merge(A, left, mid, right)

1. Create two temporary arrays:

LeftArray = A[left...mid]

RightArray = A[mid+1...right]

2. Set i = 0 (index for LeftArray)

3. Set j = 0 (index for RightArray)

4. Set k = left (index for merged array)

5. While i < size of LeftArray AND j < size of RightArray:

 If LeftArray[i] ≤ RightArray[j]:

 A[k] = LeftArray[i]

 i = i + 1

 Else:

 A[k] = RightArray[j]

 j = j + 1

 k = k + 1

6. Copy remaining elements of LeftArray (if any) into A

7. Copy remaining elements of RightArray (if any) into A

5.Merge Sort Algorithm (Divide and Conquer)

Step 1: Divide

[38, 27, 43, 3] | [9, 82, 10]

Step 2: Further Divide

[38, 27] | [43, 3] | [9, 82] | [10]

Step 3: Base Case Reached (Single Elements)

[38] | [27] | [43] | [3] | [9] | [82] | [10]

Step 4: Merge Pairs

[27, 38] | [3, 43] | [9, 82] | [10]

Step 5: Merge Again

[3, 27, 38, 43] | [9, 10, 82]

Step 6: Final Merge

[3, 9, 10, 27, 38, 43, 82] (Sorted Array)

5.Merge Sort Algorithm (Divide and Conquer)

Merge Sort Complexity:

- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

Applications of Merge Sort:

- Sorting linked lists (since it doesn't require random access like QuickSort).
- External sorting (e.g., sorting huge datasets that don't fit into RAM).
- Stable sorting (preserving the order of equal elements).

- **Steps of Quick Sort:**
- Pick a **pivot** (typically the last or first element).
- Partition the array:
 - Move smaller elements to the left of the pivot.
 - Move larger elements to the right of the pivot.
- Recursively apply Quick Sort on the left and right partitions.

- **Advantages of Quick Sort**
- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.
- It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.
- Fastest general purpose algorithm for large data when stability is not required.

- **Disadvantages of Quick Sort**
- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

• Applications of Quick Sort

- Efficient for sorting large datasets with $O(n \log n)$ average-case time complexity.
- Used in partitioning problems like finding the k th smallest element or dividing arrays by pivot.
- Integral to randomized algorithms, offering better performance than deterministic approaches.
- Applied in cryptography for generating random permutations and unpredictable encryption keys.
- Partitioning step can be parallelized for improved performance in multi-core or distributed systems.
- Important in theoretical computer science for analyzing average-case complexity and developing new techniques.

5. Picking Pivot Element

- Always pick the first (or last) element as a pivot. The below implementation picks the last element as pivot. The problem with this approach is it ends up in the worst case when array is already sorted.
- Pick a random element as a pivot. This is a preferred approach because it does not have a pattern for which the worst case happens.
- Pick the median element is pivot. This is an ideal approach in terms of time complexity as we can find median in linear time and the partition function will always divide the input array into two halves. But it takes more time on average as median finding has high constants.

5. Quick Sort Example

- Example: Sorting [10, 5, 2, 8, 7, 3]

Step 1: Choose Pivot

We pick the last element as the pivot: 3

Step 2: Partition the Array

- Left Partition (≤ 3): [2]
- Pivot: [3]
- Right Partition (> 3): [10, 5, 8, 7]

Now, recursively sort the left and right partitions.

Step 3: Sort Left Partition [2]

Since it has only one element, it remains [2].

5. Quick Sort Example

Step 4: Sort Right Partition [10, 5, 8, 7]

- Pivot = 7
- Left Partition (≤ 7): [5]
- Pivot: [7]
- Right Partition (> 7): [10, 8]

Now, sort [5] (already sorted) and [10, 8].

Step 5: Sort Right Partition [10, 8]

- Pivot = 8
- Left Partition (≤ 8): [] (empty)
- Pivot: [8]
- Right Partition (> 8): [10]

Sorted result: [8, 10]

Step 6: Merge Results

- [2] + [3] + [5] + [7] + [8, 10]
- Final sorted array: [2, 3, 5, 7, 8, 10]

Topological sorting is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge ($U \rightarrow V$), the vertex U appears before V in the ordering.

In simple terms:

- It arranges nodes sequentially while ensuring that dependencies (edges) are respected.
- It is only possible in DAGs (graphs without cycles).

Steps to be followed for Topological sorting

- **DFS Exploration Order:** Explain that DFS visits deeper nodes first.
- **Stack Push Order:** Show that nodes are pushed to the stack only after all their dependencies are resolved.
- **Reversing the Stack:** Emphasize that the correct topological order is obtained by popping elements

• $A \rightarrow C \rightarrow E$



$B \quad D \rightarrow F$

Step 1: Compute In-Degrees

Each node's in-degree (number of incoming edges):

Node In-Degree

A 0

B 1 (from A)

C 1 (from A)

D 1 (from C)

E 1 (from C)

F 1 (from D)

1. Start from A (no incoming edges).
2. Explore B, then push it to the stack.
3. Explore C, then go to $D \rightarrow F$, pushing F and D in order.
4. Explore E, then push E to the stack.
5. Push C, then push A at the end.
6. Final stack (bottom to top): $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$

Step 3: Stack at the End (Bottom to Top)

$A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$

Step 4: Obtain the Final Topological Order

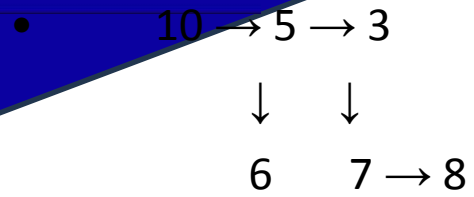
After popping the stack, the correct Topological Order is:

$A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$



Final Answer (DFS-Based Topological Sorting):

$A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$



Step 1: Identify In-Degrees

Each node's in-degree (number of incoming edges):

$10 \rightarrow 0$ (no incoming edges)

$5 \rightarrow 1$ (from 10)

$3 \rightarrow 1$ (from 5)

$6 \rightarrow 1$ (from 10)

$7 \rightarrow 1$ (from 5)

$8 \rightarrow 1$ (from 7)

1. Start DFS from 10
2. Visit 5 $\rightarrow$ Visit 3 $\rightarrow$ Push 3 to stack
3. Visit 7 $\rightarrow$ Visit 8 $\rightarrow$ Push 8 to stack
4. Push 7, then 5 to stack
5. Visit 6 from 10 $\rightarrow$ Push 6 to stack
6. Push 10 to stack (as all dependencies are processed)

- Step 3: Obtain the Topological Order

Final Stack (top to bottom):

$10 \rightarrow 6 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 3$

Final Topological Sorting (Popping Stack):

$10 \rightarrow 6 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 3$

- • **Why Do We Reverse the Stack?**
- The **first element** pushed into the stack is the **last to be completed** in DFS.
- The **last element** pushed is the **first one that should appear** in topological order.
- **Reversing the stack** ensures that dependencies appear **before** dependents.
- Thus, when we print the stack in reverse order, we get the correct **topological sort order**.

- The **Master Theorem** is a powerful tool used to solve **recurrence relations** that arise in the analysis of **divide-and-conquer algorithms**. It provides a systematic way to determine the time complexity of recursive algorithms of the form:
- We use the **Master Theorem** to quickly find the **time complexity** of **divide-and-conquer problems**. It applies to recurrence relations of this type:
 - $$T(n) = aT(n/b) + f(n)$$
 - **where** n = Input size
 - a = Number of subproblems
 - b = How much each subproblem reduces (factor)
 - $f(n)$ = Work done outside recursion (splitting/merging)

Three Simple Cases

1. If grows **slower** $\rightarrow$ Answer = $O(n^{\lfloor \log_b a \rfloor})$
2. If grows **at the same speed** $\rightarrow$ Answer = $O(n^{\lfloor \log_b a \rfloor} \log n)$
3. If grows **faster** $\rightarrow$ Answer = $O(f(n))$

- **Examples (Step-by-Step)**
- **Example 1:**
- $T(n) = 4T(n/2) + n$
- $a = 4, b = 2, f(n) = n$
- Compute $\log_2 4 = 2$ (since $2^2 = 4$)
- Compare $f(n) = n$ with n^2
 - n is smaller than n^2
- **Use Case 1 $\rightarrow$ Answer: $O(n^2)$**

- **Example 2:**
- **$T(n) = 2T(n/3) + n$**
- **$a = 2, b = 3, f(n) = n$**
- **Compute $\log_3 2 \approx 0.63$**
- **Compare $f(n) = n$ with $n^{0.63}$**
 - **n is bigger than $n^{0.63}$**
- **Use Case 3 $\rightarrow$ Answer: $O(n)$**

- The recurrence relation for Merge Sort is:
$$T(n) = 2T(n/2) + O(n)$$
- where:
- $a = 2 \rightarrow$ The problem is divided into **two** subproblems.
- $b = 2 \rightarrow$ Each subproblem is of size $n/2$.
- $f(n) = O(n) \rightarrow$ The merging step takes **linear** time.

- **Step 1: Compute $\log_b a$**
- $\log_2 2 = 1$
- **Step 2: Compare $f(n)$ with $n^{(\log_b a)}$**
- $f(n) = O(n)$
 $n^{(\log_2 2)} = n^1 = O(n)$
- Since $f(n)$ is equal to $O(n^{(\log_b a)})$,
- we apply **Case 2** of the **Master Theorem**, which gives:
 $T(n) = O(n \log n)$
- **Final Answer:**
- **Merge Sort runs in $O(n \log n)$ time complexity**

